

Process & Decision Documentation

Project/Assignment Decisions

Throughout Side Quest Week 4 I led the decision making, and cleanup of the code while employing AI to create the code. I decided to integrate the inclusion of spikes in the third level and AI wrote the code that allowed me to do that. I made the decision to add spikes, to raise the stakes of the game. The effect this had was making the game more dynamic, interesting, and difficult to complete.

Role-Based Process Evidence

If GenAI was used to support planning, coordination, or role management, this should be noted briefly here as part of the process evidence. Deeper reflection on how GenAI affected teamwork, fairness, and coordination over time belongs in the GenAI Reflections (A4).

Entry Header

Name: Fiffy Olafenwa

Role(s): Code cleaner and decision maker

Primary responsibility for this work: Editing the code output from AI and ideation of spike addition.

Goal of Work Session

Create a new level using AI including the obstacle, spikes.

GenAI Documentation

Date Used: February 8th, 2026

Tool Disclosure: Microsoft 365 Copilot

Purpose of Use: I used AI for code creation and structure.

Summary of Interaction: The tool contributed by generating code I could use to replace and add to the example code in order to add spikes and a new level to the game.

Human Decision Point(s): I added on the necessary code from AI such as the new level code and the spike function. I also added spikes to the previous levels to improve consistency. I also made the decision of not including a function to randomly generate a new level each time as I felt it was too intricate for the style of the game.

Integrity & Verification Note: I check the GenAI for appropriateness, bias, and incorrectness by carefully reading through and selecting the code that it gave me.

Scope of GenAI Use: The idea of adding spikes was thought of by me. I edited the previous level Json's to add spikes myself. The new spike's function and new level was generated completely by Microsoft Copilot with slight edits from me. I also changed the letter spacing of the new text for the instructions and changed the colour to white myself.

Limitations or Misfires: The tool did miss some spacing issues when coming up with the new instructions. The tool also added spikes in areas that would make completing the game impossible by placing them right in front of the goal.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

- I decided to include spikes in the new level
- I then copied each Js and Json code into Microsoft Copilot
- I then asked GenAI to create the new level code and add spikes
- Inserted the code into the example 4 template
- Code cleanup and slight editing
- Inserted the new screen code into visual studio code
- Play tested the game and realized the “home” button was not working
- Moments of uncertainty, failure, or rework: I reworked portions of the AI output including changing spike position, and omitting the addition of code that would add levels procedurally.

Decision Points & Trade-offs

The key decision I made was to omit the code for procedurally generating a level as I felt it would be too complicated for the game style, particularly because the spike additions would be too randomized and might make the game unplayable.

Verification & Judgement

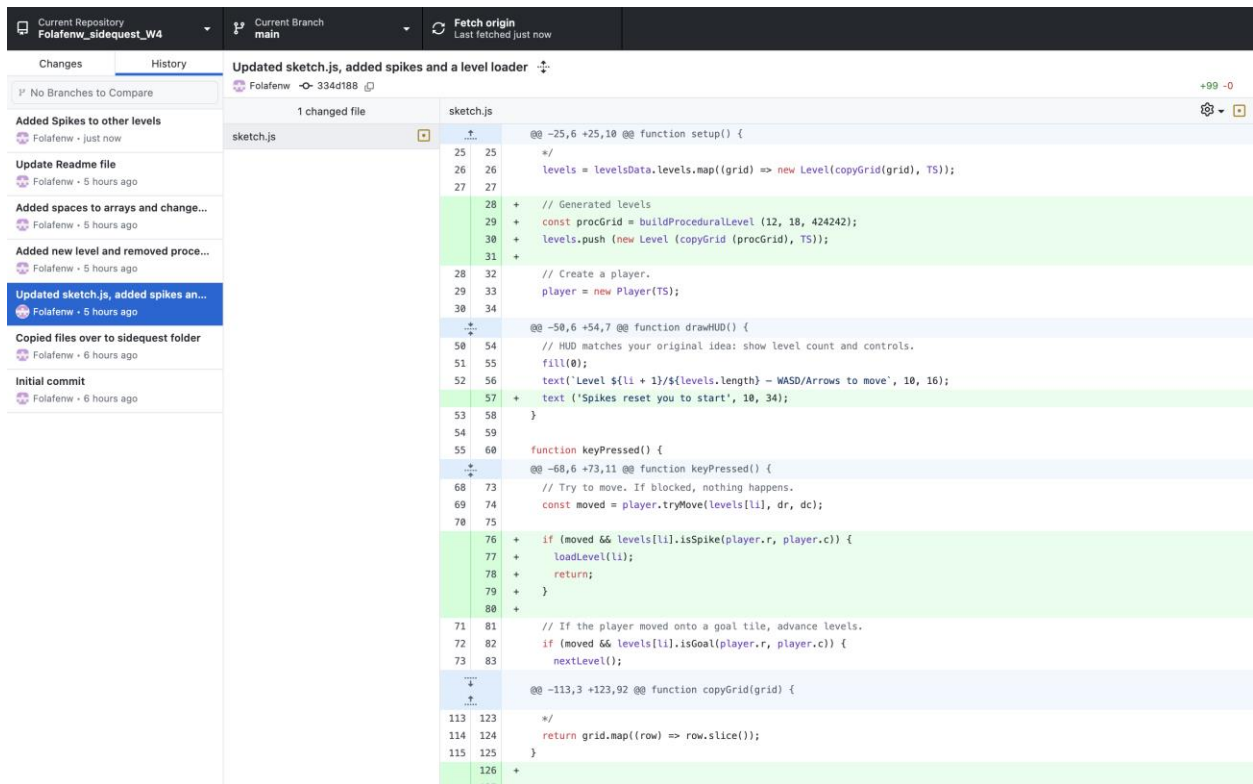
I evaluated by playtesting which is how I caught mistakes and missteps. After most changes I played the game in live server to see what it looked like and if I and the AI was on the right track.

Limitations, Dead Ends, or Open Questions

The AI although very helpful, added some spike sin places in front of the code and also seemed to have an issue with text spacing. This shows that AI can have an issue with game logic and making sure things are playable.

Appendix

Github Screenshots



The screenshot shows a GitHub commit page for the repository 'Folafemw_sidequest_W4'. The commit is titled 'Updated sketch.js, added spikes and a level loader' and is attributed to 'Folafemw' with a commit hash of '334d188'. The commit message is 'Updated sketch.js, added spikes and a level loader'. The commit is on the 'main' branch. The commit is dated '5 hours ago'. The commit is a 'New commit'.

The commit shows a single file change: 'sketch.js'. The file is 126 lines long. The commit shows the following code changes:

```
@@ -25,6 +25,10 @@ function setup() {  
25 25  */  
26 26  levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));  
27 27  
28 +  // Generated levels  
29 +  const procGrid = buildProceduralLevel(12, 18, 424242);  
30 +  levels.push(new Level(copyGrid(procGrid), TS));  
31 +  
28 32  // Create a player.  
29 33  player = new Player(TS);  
30 34  
@@ -50,6 +54,7 @@ function drawHUD() {  
50 54  // HUD matches your original idea: show level count and controls.  
51 55  fill(0);  
52 56  text('Level ${(li + 1)/${levels.length} - WASD/Arrows to move', 10, 16);  
57 +  text('Spikes reset you to start', 10, 34);  
53 58  }  
54 59  
55 60  function keyPressed() {  
@@ -68,73 +68,11 @@ function keyPressed() {  
68 73  // Try to move. If blocked, nothing happens.  
69 74  const moved = player.tryMove(levels[li], dr, dc);  
70 75  
76 +  if (moved && levels[li].isSpike(player.r, player.c)) {  
77 +  loadLevel(li);  
78 +  return;  
79 +  }  
80 +  
71 81  // If the player moved onto a goal tile, advance levels.  
72 82  if (moved && levels[li].isGoal(player.r, player.c)) {  
73 83  nextLevel();  
@@ -113,3 +123,92 @@ function copyGrid(grid) {  
113 123  */  
114 124  return grid.map((row) => row.slice());  
115 125  }  
126 +  
127 +
```

Current Repository
Folafernw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes

History

No Branches to Compare

Added Spikes to other levels
Folafernw · 1 minute ago

Update Readme file
Folafernw · 5 hours ago

Added spaces to arrays and change...
Folafernw · 5 hours ago

Added new level and removed proce...
Folafernw · 5 hours ago

Updated sketch.js, added spikes an...
Folafernw · 5 hours ago

Copied files over to sidequest folder
Folafernw · 6 hours ago

Initial commit
Folafernw · 6 hours ago

Updated sketch.js, added spikes and a level loader

Folafernw · 334d188

1 changed file

sketch.js

+99 -0

sketch.js

113 123
114 124
115 125

```
113 123      */  
114 124      return grid.map((row) => row.slice());  
115 125  }  
  
126  +  
127  +  
128  + function buildProceduralLevel(rows, cols, seed = 12345) {  
129  +   // Safety: minimum viable size with outer walls  
130  +   rows = Math.max(rows, 7);  
131  +   cols = Math.max(cols, 10);  
132  +  
133  +   // 1) Start with all walls  
134  +   const G = Array.from({ length: rows }, () => Array(cols).fill(1));  
135  +  
136  +   // 2) Carve interior floors with a double loop  
137  +   for (let r = 1; r < rows - 1; r++) {  
138  +     for (let c = 1; c < cols - 1; c++) {  
139  +       G[r][c] = 0; // floor  
140  +     }  
141  +   }  
142  +  
143  +   // 3) Place start & goal  
144  +   const start = { r: 1, c: 1 };  
145  +   const goal = { r: rows - 2, c: cols - 2 };  
146  +   G[start.r][start.c] = 2;  
147  +   G[goal.r][goal.c] = 3;  
148  +  
149  +   // 4) Add some internal wall "pillars" using loops  
150  +   //   (simple columns to create navigation choices)  
151  +   for (let r = 2; r < rows - 2; r++) {  
152  +     if (r % 2 === 0) {  
153  +       const c1 = Math.floor(cols * 0.33);  
154  +       const c2 = Math.floor(cols * 0.66);  
155  +       G[r][c1] = 1;  
156  +       G[r][c2] = 1;  
157  +     }  
158  +   }  
159  +  
160  +   // 5) Spike patterns with loops
```

Current Repository
Folafernw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes

History

No Branches to Compare

Added Spikes to other levels
Folafernw · 1 minute ago

Update Readme file
Folafernw · 5 hours ago

Added spaces to arrays and change...
Folafernw · 5 hours ago

Added new level and removed proce...
Folafernw · 5 hours ago

Updated sketch.js, added spikes an...
Folafernw · 5 hours ago

Copied files over to sidequest folder
Folafernw · 6 hours ago

Initial commit
Folafernw · 6 hours ago

Updated sketch.js, added spikes and a level loader

Folafernw · 334d188

1 changed file

sketch.js

+99 -0

sketch.js

160 +
161 +
162 +
163 +
164 +
165 +
166 +
167 +
168 +
169 +
170 +
171 +
172 +
173 +
174 +
175 +
176 +
177 +
178 +
179 +
180 +
181 +
182 +
183 +
184 +
185 +
186 +
187 +
188 +
189 +
190 +
191 +
192 +
193 +
194 +
195 +
196 +
197 +

```
160  +   // 5) Spike patterns with loops  
161  +  
162  +   // 5a) Checkerboard spikes in a band (every 2 tiles)  
163  +   for (let r = 2; r < rows - 2; r++) {  
164  +     for (let c = 2; c < cols - 2; c++) {  
165  +       if ((r + c) % 2 === 0 && r >= Math.floor(rows * 0.35) && r <= Math.floor(rows * 0.65)) {  
166  +         if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;  
167  +       }  
168  +     }  
169  +   }  
170  +  
171  +   // 5b) A vertical "spike line" on every 3rd column  
172  +   for (let c = 3; c < cols - 3; c += 3) {  
173  +     for (let r = 2; r < rows - 2; r++) {  
174  +       if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;  
175  +     }  
176  +   }  
177  +  
178  +   // 5c) Sprinkle some random spikes (deterministic with a seed)  
179  +   const rnd = mulberry32(seed);  
180  +   let sprinkled = Math.floor((rows * cols) * 0.08);  
181  +   while (sprinkled > 0) {  
182  +     const r = 1 + Math.floor(rnd() * (rows - 2));  
183  +     const c = 1 + Math.floor(rnd() * (cols - 2));  
184  +     if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) {  
185  +       G[r][c] = 4;  
186  +       sprinkled--;  
187  +     }  
188  +   }  
189  +  
190  +   // Optional: ensure at least a horizontal corridor from start side  
191  +   // (reduce frustration; keeps a guaranteed safe band)  
192  +   const safeRow = start.r + 1;  
193  +   for (let c = start.c; c < cols - 1; c++) {  
194  +     if (!isStartOrGoal(start, goal, safeRow, c) && G[safeRow][c] !== 1) {  
195  +       G[safeRow][c] = 0; // clear any spikes from this band  
196  +     }  
197  +   }
```

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes

History

No Branches to Compare

Added Spikes to other levels

Folafenw • 1 minute ago

Update Readme file

Folafenw • 5 hours ago

Added spaces to arrays and change...

Folafenw • 5 hours ago

Added new level and removed proce...

Folafenw • 5 hours ago

Updated sketch.js, added spikes an...

Folafenw • 5 hours ago

Copied files over to sidequest folder

Folafenw • 6 hours ago

Initial commit

Folafenw • 6 hours ago

Updated sketch.js, added spikes and a level loader

Folafenw • 334d188

+99 -0

1 changed file

sketch.js

sketch.js

177 +
178 +
179 +
180 +
181 +
182 +
183 +
184 +
185 +
186 +
187 +
188 +
189 +
190 +
191 +
192 +
193 +
194 +
195 +
196 +
197 +
198 +
199 +
200 +
201 +
202 +
203 +
204 +
205 +
206 +
207 +
208 +
209 +
210 +
211 +
212 +
213 +
214 +

+
// 5c) Sprinkle some random spikes (deterministic with a seed)
const rnd = mulberry32(seed);
let sprinkled = Math.floor((rows * cols) * 0.00);
while (sprinkled > 0) {
const r = 1 + Math.floor(rnd() * (rows - 2));
const c = 1 + Math.floor(rnd() * (cols - 2));
if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) {
G[r][c] = 4;
sprinkled--;
}
}

// Optional: ensure at least a horizontal corridor from start side
// (reduce frustration; keeps a guaranteed safe band)
const safeRow = start.r + 1;
for (let c = start.c; c < cols - 1; c++) {
if (!isStartOrGoal(start, goal, safeRow, c) && G[safeRow][c] !== 1) {
G[safeRow][c] = 0; // clear any spikes from this band
}
}
return G;
}

function isStartOrGoal(start, goal, r, c) {
return (r === start.r && c === start.c) || (r === goal.r && c === goal.c);
}

// Deterministic RNG for reproducible procedural levels
function mulberry32(a) {
return function () {
let t = a += 0x602B79F5;
t = Math.imul(t ^ (t >> 15), t | 1);
t ^= t + Math.imul(t ^ (t >> 7), t | 61);
return ((t ^ (t >> 14)) >> 0) / 4294967296;
};
}

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 1 minute ago

Changes

History

No Branches to Compare

Added Spikes to other levels

Folafenw • 1 minute ago

Update Readme file

Folafenw • 5 hours ago

Added spaces to arrays and change...

Folafenw • 5 hours ago

Added new level and removed proce...

Folafenw • 5 hours ago

Updated sketch.js, added spikes an...

Folafenw • 5 hours ago

Copied files over to sidequest folder

Folafenw • 6 hours ago

Initial commit

Folafenw • 6 hours ago

Added new level and removed procedure level building code

Folafenw • af685a6

+31 -93

3 changed files

level.js

level.js

level.js

sketch.js

level.js

11 +
12 +
13 +
14 +
15 +
16 +
17 +
18 +
19 +
20 +
21 +
22 +
23 +
24 +
25 +
26 +
27 +
28 +
29 +
30 +
31 +
32 +
33 +
34 +
35 +
36 +
37 +
38 +
39 +
40 +
41 +
42 +
43 +
44 +
45 +
46 +
47 +
48 +
49 +
50 +
51 +
52 +
53 +
54 +
55 +
56 +
57 +
58 +
59 +
60 +
61 +
62 +
63 +
64 +
65 +
66 +
67 +
68 +
69 +
70 +
71 +
72 +
73 +
74 +
75 +
76 +
77 +
78 +
79 +
80 +
81 +
82 +
83 +
84 +
85 +
86 +
87 +
88 +
89 +
90 +
91 +
92 +
93 +
94 +
95 +
96 +
97 +
98 +
99 +
100 +
101 +
102 +
103 +
104 +
105 +
106 +
107 +
108 +
109 +
110 +
111 +
112 +
113 +
114 +
115 +
116 +
117 +
118 +
119 +
120 +
121 +
122 +
123 +
124 +
125 +
126 +
127 +
128 +
129 +
130 +
131 +
132 +
133 +
134 +
135 +
136 +
137 +
138 +
139 +
140 +
141 +
142 +
143 +
144 +
145 +
146 +
147 +
148 +
149 +
150 +
151 +
152 +
153 +
154 +
155 +
156 +
157 +
158 +
159 +
160 +
161 +
162 +
163 +
164 +
165 +
166 +
167 +
168 +
169 +
170 +
171 +
172 +
173 +
174 +
175 +
176 +
177 +
178 +
179 +
180 +
181 +
182 +
183 +
184 +
185 +
186 +
187 +
188 +
189 +
190 +
191 +
192 +
193 +
194 +
195 +
196 +
197 +
198 +
199 +
200 +
201 +
202 +
203 +
204 +
205 +
206 +
207 +
208 +
209 +
210 +
211 +
212 +
213 +
214 +
215 +
216 +
217 +
218 +
219 +
220 +
221 +
222 +
223 +
224 +
225 +
226 +
227 +
228 +
229 +
230 +
231 +
232 +
233 +
234 +
235 +
236 +
237 +
238 +
239 +
240 +
241 +
242 +
243 +
244 +
245 +
246 +
247 +
248 +
249 +
250 +
251 +
252 +
253 +
254 +
255 +
256 +
257 +
258 +
259 +
260 +
261 +
262 +
263 +
264 +
265 +
266 +
267 +
268 +
269 +
270 +
271 +
272 +
273 +
274 +
275 +
276 +
277 +
278 +
279 +
280 +
281 +
282 +
283 +
284 +
285 +
286 +
287 +
288 +
289 +
290 +
291 +
292 +
293 +
294 +
295 +
296 +
297 +
298 +
299 +
300 +
301 +
302 +
303 +
304 +
305 +
306 +
307 +
308 +
309 +
310 +
311 +
312 +
313 +
314 +
315 +
316 +
317 +
318 +
319 +
320 +
321 +
322 +
323 +
324 +
325 +
326 +
327 +
328 +
329 +
330 +
331 +
332 +
333 +
334 +
335 +
336 +
337 +
338 +
339 +
340 +
341 +
342 +
343 +
344 +
345 +
346 +
347 +
348 +
349 +
350 +
351 +
352 +
353 +
354 +
355 +
356 +
357 +
358 +
359 +
360 +
361 +
362 +
363 +
364 +
365 +
366 +
367 +
368 +
369 +
370 +
371 +
372 +
373 +
374 +
375 +
376 +
377 +
378 +
379 +
380 +
381 +
382 +
383 +
384 +
385 +
386 +
387 +
388 +
389 +
390 +
391 +
392 +
393 +
394 +
395 +
396 +
397 +
398 +
399 +
400 +
401 +
402 +
403 +
404 +
405 +
406 +
407 +
408 +
409 +
410 +
411 +
412 +
413 +
414 +
415 +
416 +
417 +
418 +
419 +
420 +
421 +
422 +
423 +
424 +
425 +
426 +
427 +
428 +
429 +
430 +
431 +
432 +
433 +
434 +
435 +
436 +
437 +
438 +
439 +
440 +
441 +
442 +
443 +
444 +
445 +
446 +
447 +
448 +
449 +
450 +
451 +
452 +
453 +
454 +
455 +
456 +
457 +
458 +
459 +
460 +
461 +
462 +
463 +
464 +
465 +
466 +
467 +
468 +
469 +
470 +
471 +
472 +
473 +
474 +
475 +
476 +
477 +
478 +
479 +
480 +
481 +
482 +
483 +
484 +
485 +
486 +
487 +
488 +
489 +
490 +
491 +
492 +
493 +
494 +
495 +
496 +
497 +
498 +
499 +
500 +
501 +
502 +
503 +
504 +
505 +
506 +
507 +
508 +
509 +
510 +
511 +
512 +
513 +
514 +
515 +
516 +
517 +
518 +
519 +
520 +
521 +
522 +
523 +
524 +
525 +
526 +
527 +
528 +
529 +
530 +
531 +
532 +
533 +
534 +
535 +
536 +
537 +
538 +
539 +
540 +
541 +
542 +
543 +
544 +
545 +
546 +
547 +
548 +
549 +
550 +
551 +
552 +
553 +
554 +
555 +
556 +
557 +
558 +
559 +
560 +
561 +
562 +
563 +
564 +
565 +
566 +
567 +
568 +
569 +
570 +
571 +
572 +
573 +
574 +
575 +
576 +
577 +
578 +
579 +
580 +
581 +
582 +
583 +
584 +
585 +
586 +
587 +
588 +
589 +
590 +
591 +
592 +
593 +
594 +
595 +
596 +
597 +
598 +
599 +
600 +
601 +
602 +
603 +
604 +
605 +
606 +
607 +
608 +
609 +
610 +
611 +
612 +
613 +
614 +
615 +
616 +
617 +
618 +
619 +
620 +
621 +
622 +
623 +
624 +
625 +
626 +
627 +
628 +
629 +
630 +
631 +
632 +
633 +
634 +
635 +
636 +
637 +
638 +
639 +
640 +
641 +
642 +
643 +
644 +
645 +
646 +
647 +
648 +
649 +
650 +
651 +
652 +
653 +
654 +
655 +
656 +
657 +
658 +
659 +
660 +
661 +
662 +
663 +
664 +
665 +
666 +
667 +
668 +
669 +
670 +
671 +
672 +
673 +
674 +
675 +
676 +
677 +
678 +
679 +
680 +
681 +
682 +
683 +
684 +
685 +
686 +
687 +
688 +
689 +
690 +
691 +
692 +
693 +
694 +
695 +
696 +
697 +
698 +
699 +
700 +
701 +
702 +
703 +
704 +
705 +
706 +
707 +
708 +
709 +
710 +
711 +
712 +
713 +
714 +
715 +
716 +
717 +
718 +
719 +
720 +
721 +
722 +
723 +
724 +
725 +
726 +
727 +
728 +
729 +
730 +
731 +
732 +
733 +
734 +
735 +
736 +
737 +
738 +
739 +
740 +
741 +
742 +
743 +
744 +
745 +
746 +
747 +
748 +
749 +
750 +
751 +
752 +
753 +
754 +
755 +
756 +
757 +
758 +
759 +
760 +
761 +
762 +
763 +
764 +
765 +
766 +
767 +
768 +
769 +
770 +
771 +
772 +
773 +
774 +
775 +
776 +
777 +
778 +
779 +
780 +
781 +
782 +
783 +
784 +
785 +
786 +
787 +
788 +
789 +
790 +
791 +
792 +
793 +
794 +
795 +
796 +
797 +
798 +
799 +
800 +
801 +
802 +
803 +
804 +
805 +
806 +
807 +
808 +
809 +
810 +
811 +
812 +
813 +
814 +
815 +
816 +
817 +
818 +
819 +
820 +
821 +
822 +
823 +
824 +
825 +
826 +
827 +
828 +
829 +
830 +
831 +
832 +
833 +
834 +
835 +
836 +
837 +
838 +
839 +
840 +
841 +
842 +
843 +
844 +
845 +
846 +
847 +
848 +
849 +
850 +
851 +
852 +
853 +
854 +
855 +
856 +
857 +
858 +
859 +
860 +
861 +
862 +
863 +
864 +
865 +
866 +
867 +
868 +
869 +
870 +
871 +
872 +
873 +
874 +
875 +
876 +
877 +
878 +
879 +
880 +
881 +
882 +
883 +
884 +
885 +
886 +
887 +
888 +
889 +
890 +
891 +
892 +
893 +
894 +
895 +
896 +
897 +
898 +
899 +
900 +
901 +
902 +
903 +
904 +
905 +
906 +
907 +
908 +
909 +
910 +
911 +
912 +
913 +
914 +
915 +
916 +
917 +
918 +
919 +
920 +
921 +
922 +
923 +
924 +
925 +
926 +
927 +
928 +
929 +
930 +
931 +
932 +
933 +
934 +
935 +
936 +
937 +
938 +
939 +
940 +
941 +
942 +
943 +
944 +
945 +
946 +
947 +
948 +
949 +
950 +
951 +
952 +
953 +
954 +
955 +
956 +
957 +
958 +
959 +
960 +
961 +
962 +
963 +
964 +
965 +
966 +
967 +
968 +
969 +
970 +
971 +
972 +
973 +
974 +
975 +
976 +
977 +
978 +
979 +
980 +
981 +
982 +
983 +
984 +
985 +
986 +
987 +
988 +
989 +
990 +
991 +
992 +
993 +
994 +
995 +
996 +
997 +
998 +
999 +
1000 +
1001 +
1002 +
1003 +
1004 +
1005 +
1006 +
1007 +
1008 +
1009 +
1010 +
1011 +
1012 +
1013 +
1014 +
1015 +
1016 +
1017 +
1018 +
1019 +
1020 +
1021 +
1022 +
1023 +
1024 +
1025 +
1026 +
1027 +
1028 +
1029 +
1030 +
1031 +
1032 +
1033 +
1034 +
1035 +
1036 +
1037 +
1038 +
1039 +
1040 +
1041 +
1042 +
1043 +
1044 +
1045 +
1046 +
1047 +
1048 +
1049 +
1050 +
1051 +
1052 +
1053 +
1054 +
1055 +
1056 +
1057 +
1058 +
1059 +
1060 +
1061 +
1062 +
1063 +
1064 +
1065 +
1066 +
1067 +
1068 +
1069 +
1070 +
1071 +
1072 +
1073 +
1074 +
1075 +
1076 +
1077 +
1078 +
1079 +
1080 +
1081 +
1082 +
1083 +
1084 +
1085 +
1086 +
1087 +
1088 +
1089 +
1090 +
1091 +
1092 +
1093 +
1094 +
1095 +
1096 +
1097 +
1098 +
1099 +
1100 +
1101 +
1102 +
1103 +
1104 +
1105 +
1106 +
1107 +
1108 +
1109 +
1110 +
1111 +
1112 +
1113 +
1114 +
1115 +
1116 +
1117 +
1118 +
1119 +
1120 +
1121 +
1122 +
1123 +
1124 +
1125 +
1126 +
1127 +
1128 +
1129 +
1130 +
1131 +
1132 +
1133 +
1134 +
1135 +
1136 +
1137 +
1138 +
1139 +
1140 +
1141 +
1142 +
1143 +
1144 +
1145 +
1146 +
1147 +
1148 +
1149 +
1150 +
1151 +
1152 +
1153 +
1154 +
1155 +
1156 +
1157 +
1158 +
1159 +
1160 +
1161 +
1162 +
1163 +
1164 +
1165 +
1166 +
1167 +
1168 +
1169 +
1170 +
1171 +
1172 +
1173 +
1174 +
1175 +
1176 +
1177 +
1178 +
1179 +
1180 +
1181 +
1182 +
1183 +
1184 +
1185 +
1186 +
1187 +
1188 +
1189 +
1190 +
1191 +
1192 +
1193 +
1194 +
1195 +
1196 +
1197 +
1198 +
1199 +
1200 +
1201 +
1202 +
1203 +
1204 +
1205 +
1206 +
1207 +
1208 +
1209 +
1210 +
1211 +
1212 +
1213 +
1214 +
1215 +
1216 +
1217 +
1218 +
1219 +
1220 +
1221 +
1222 +
1223 +
1224 +
1225 +
1226 +
1227 +
1228 +
1229 +
1230 +
1231 +
1232 +
1233 +
1234 +
1235 +
1236 +
1237 +
1238 +
1239 +
1240 +
1241 +
1242 +
1243 +
1244 +
1245 +
1246 +
1247 +
1248 +
1249 +
1250 +
1251 +
1252 +
1253 +
1254 +
1255 +
1256 +
1257 +
1258 +
1259 +
1260 +
1261 +
1262 +
1263 +
1264 +
1265 +
1266 +
1267 +
1268 +
1269 +
1270 +
1271 +
1272 +
1273 +
1274 +
1275 +
1276 +
1277 +
1278 +
1279 +
1280 +
1281 +
1282 +
1283 +
1284 +
1285 +
1286 +
1287 +
1288 +
1289 +
1290 +
1291 +
1292 +
1293 +
1294 +
1295 +
1296 +
1297 +
1298 +
1299 +
1300 +
1301 +
1302 +
1303 +
1304 +
1305 +
1306 +
1307 +
1308 +
1309 +
1310 +
1311 +
1312 +
1313 +
1314 +
1315 +
1316 +
1317 +
1318 +
1319 +
1320 +
1321 +
1322 +
1323 +
1324 +
1325 +
1326 +
1327 +
1328 +
1329 +
1330 +
1331 +
1332 +
1333 +
1334 +
1335 +
1336 +
1337 +
1338 +
1339 +
1340 +
1341 +
1342 +
1343 +
1344 +
1345 +
1346 +
1347 +
1348 +
1349 +
1350 +
1351 +
1352 +
1353 +
1354 +
1355 +
1356 +
1357 +
1358 +
1359 +
1360 +
1361 +
1362 +
1363 +
1364 +
1365 +
1366 +
1367 +
1368 +
1369 +
1370 +
1371 +
1372 +
1373 +
1374 +
1375 +
1376 +
1377 +
1378 +
1379 +
1380 +
1381 +
1382 +
1383 +
1384 +
1385 +
1386 +
1387 +
1388 +
1389 +
1390 +
1391 +
1392 +
1393 +
1394 +
1395 +
1396 +
1397 +
1398 +
1399 +
1400 +
1401 +
1402 +
1403 +
1404 +
1405 +
1406 +
1407 +
1408 +
1409 +
1410 +
1411 +
1412 +
1413 +
1414 +
1415 +
1416 +
1417 +
1418 +
1419 +
1420 +
1421 +
1422 +
1423 +
1424 +
1425 +
1426 +
1427 +
1428 +
1429 +
1430 +
1431 +
1432 +
1433 +
1434 +
1435 +
1436 +
1437 +
1438 +
1439 +
1440 +
1441 +
1442 +
1443 +
1444 +
1445 +
1446 +
1447 +
1448 +
1449 +
1450 +
1451 +
1452 +
1453 +
1454 +
1455 +
1456 +
1457 +
1458 +
1459 +
1460 +
1461 +
1462 +
1463 +
1464 +
1465 +
1466 +
1467 +
1468 +
1469 +
1470 +
1471 +
1472 +
1473 +
1474 +
1475 +
1476 +
1477 +
1478 +
1479 +
1480 +
1481 +
1482 +
1483 +
1484 +
1485 +
1486 +
1487 +
1488 +
1489 +
1490 +
1491 +
1492 +
1493 +
1494 +
1495 +
1496 +
1497 +
1498 +
1499 +
1

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 1 minute ago

ChangesHistory

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added new level and removed procedure level building code

Folafenw • af685a6

+31 -93

3 changed files

levels.json

level.js

levels.json

sketch.js

@@ -17,6 +17,16 @@
[1, 0, 0, 0, 1, 0, 0, 1, 0, 1],
[1, 1, 1, 0, 1, 0, 0, 1, 0, 1],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

20 +],
21 + [
22 + [1,1,1,1,1,1,1,1,1,1],
23 + [1,2,0,0,0,0,4,0,3,1],
24 + [1,0,1,4,1,0,1,0,1,1],
25 + [1,0,1,0,0,0,1,4,0,1],
26 + [1,0,0,0,1,0,0,1,0,1],
27 + [1,1,1,0,1,4,0,1,0,1],
28 + [1,1,1,1,1,1,1,1,1,1]

20 29]
30 +
21 31 }
22 32 }

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 2 minutes ago

ChangesHistory

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added new level and removed procedure level building code

Folafenw • af685a6

+31 -93

3 changed files

sketch.js

level.js

levels.json

sketch.js

@@ -25,10 +25,6 @@ function setup() {
*/
levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));

// Generated levels
const proceduralLevel = buildProceduralLevel(12, 18, 424242);
levels.push(new Level(copyGrid(proceduralLevel), TS));

// Create a player.
player = new Player(TS);

@@ -123,92 +119,3 @@ function copyGrid(grid) {
*/
return grid.map((row) => row.slice());
}

-
-
- function buildProceduralLevel(rows, cols, seed = 12345) {
- // Safety: minimum viable size with outer walls
- rows = Math.max(rows, 7);
- cols = Math.max(cols, 10);
-
- // 1) Start with all walls
- const G = Array.from({ length: rows }, () => Array(cols).fill(1));
-
- // 2) Carve interior floors with a double loop
- for (let r = 1; r < rows - 1; r++) {
- for (let c = 1; c < cols - 1; c++) {
- G[r][c] = 0; // floor
- }
- }
-
- // 3) Place start & goal
- const start = { r: 1, c: 1 };
- const goal = { r: rows - 2, c: cols - 2 };
- G[start.r][start.c] = 2;
- G[goal.r][goal.c] = 3;

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 2 minutes ago

Changes

History

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added new level and removed procedure level building code

+31 -93

3 changed files

sketch.js

level.js

levels.json

sketch.js

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

-

// 4) Add some internal wall "pillars" using loops

-

// (simple columns to create navigation choices)

-

for (let r = 2; r < rows - 2; r++) {

-

if (r % 2 === 0) {

-

const c1 = Math.floor(cols * 0.33);

-

const c2 = Math.floor(cols * 0.66);

-

G[r][c1] = 1;

-

G[r][c2] = 1;

-

}

-

}

-

// 5) Spike patterns with loops

-

// 5a) Checkerboard spikes in a band (every 2 tiles)

-

for (let r = 2; r < rows - 2; r++) {

-

for (let c = 2; c < cols - 2; c++) {

-

if ((r + c) % 2 === 0 && r >= Math.floor(rows * 0.35) && r <= Math.floor(rows * 0.65)) {

-

if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;

-

}

-

}

-

}

-

// 5b) A vertical "spike line" on every 3rd column

-

for (let c = 3; c < cols - 3; c += 3) {

-

for (let r = 2; r < rows - 2; r++) {

-

if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;

-

}

-

}

-

}

-

// 5c) Sprinkle some random spikes (deterministic with a seed)

-

const rnd = mulberry32(seed);

-

let sprinkled = Math.floor((rows * cols) * 0.08);

-

while (sprinkled > 0) {

-

const r = 1 + Math.floor(rnd() * (rows - 2));

-

const c = 1 + Math.floor(rnd() * (cols - 2));

-

if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) {

-

G[r][c] = 4;

-

sprinkled--;

-

}

-

}

-

// Optional: ensure at least a horizontal corridor from start side

-

// (reduce frustration; keeps a guaranteed safe band)

-

const safeRow = start.r + 1;

-

for (let c = start.c; c < cols - 1; c++) {

-

if (!isStartOrGoal(start, goal, safeRow, c) && G[safeRow][c] !== 1) {

-

G[safeRow][c] = 0; // clear any spikes from this band

-

}

-

}

-

return G;

-

}

-

function isStartOrGoal(start, goal, r, c) {

-

return (r === start.r && c === start.c) || (r === goal.r && c === goal.c);

-

}

-

// Deterministic RNG for reproducible procedural levels

-

function mulberry32(a) {

-

return function () {

-

let t = a += 0x6D2879F5;

-

t = Math.imul(t ^ (t >>> 15), t | 1);

-

t ^= t + Math.imul(t ^ (t >>> 7), t | 61);

-

return ((t ^ (t >>> 14)) >>> 0) / 4294967296;

-

};

-

}

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 2 minutes ago

Changes

History

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added new level and removed procedure level building code

+31 -93

3 changed files

sketch.js

level.js

levels.json

sketch.js

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

-

// 5c) Sprinkle some random spikes (deterministic with a seed)

-

const rnd = mulberry32(seed);

-

let sprinkled = Math.floor((rows * cols) * 0.08);

-

while (sprinkled > 0) {

-

const r = 1 + Math.floor(rnd() * (rows - 2));

-

const c = 1 + Math.floor(rnd() * (cols - 2));

-

if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) {

-

G[r][c] = 4;

-

sprinkled--;

-

}

-

}

-

// Optional: ensure at least a horizontal corridor from start side

-

// (reduce frustration; keeps a guaranteed safe band)

-

const safeRow = start.r + 1;

-

for (let c = start.c; c < cols - 1; c++) {

-

if (!isStartOrGoal(start, goal, safeRow, c) && G[safeRow][c] !== 1) {

-

G[safeRow][c] = 0; // clear any spikes from this band

-

}

-

}

-

return G;

-

}

-

function isStartOrGoal(start, goal, r, c) {

-

return (r === start.r && c === start.c) || (r === goal.r && c === goal.c);

-

}

-

// Deterministic RNG for reproducible procedural levels

-

function mulberry32(a) {

-

return function () {

-

let t = a += 0x6D2879F5;

-

t = Math.imul(t ^ (t >>> 15), t | 1);

-

t ^= t + Math.imul(t ^ (t >>> 7), t | 61);

-

return ((t ^ (t >>> 14)) >>> 0) / 4294967296;

-

};

-

}

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 2 minutes ago

ChangesHistory

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added spaces to arrays and changed text colour and height to fix readabi... ⚙

~lity

Folafenw cb58aff

+9 -0

2 changed files

sketch.js

levels.json

sketch.js

@@ -48,9 +48,9 @@ function draw() {
48 48
49 49 function drawHUD() {
50 50 // HUD matches your original idea: show level count and controls.
51 - fill(0);
51 + fill(255);
52 52 text('Level \${(li + 1)/\$(levels.length) - WASD/Arrows to move', 10, 16);
53 - text ('Spikes reset you to start', 10, 34);
53 + text ('Spikes reset you to start', 10, 30);
54 54 }
55 55
56 56 function keyPressed() {
.....
+
.....
+

Current Repository
Folafenw_sidequest_W4

Current Branch
main

Fetch origin
Last fetched 2 minutes ago

ChangesHistory

No Branches to Compare

Added Spikes to other levels
Folafenw • 2 minutes ago

Update Readme file
Folafenw • 5 hours ago

Added spaces to arrays and change...
Folafenw • 5 hours ago

Added new level and removed proce...
Folafenw • 5 hours ago

Updated sketch.js, added spikes an...
Folafenw • 5 hours ago

Copied files over to sidequest folder
Folafenw • 6 hours ago

Initial commit
Folafenw • 6 hours ago

Added Spikes to other levels ⚙

~lity

Folafenw 3e3068e

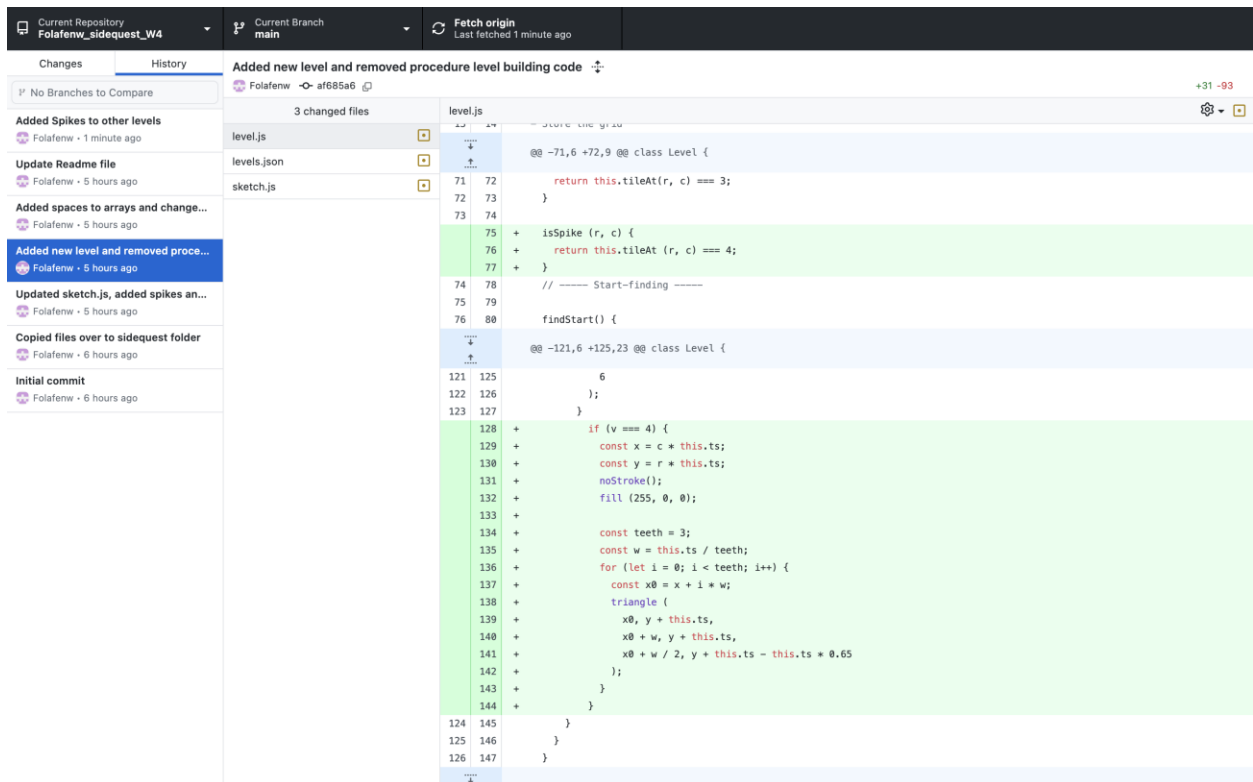
+4 -4

1 changed file

levels.json

levels.json

@@ -2,9 +2,9 @@
2 2 "levels": [
3 3 [
4 4 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
5 - [1, 2, 0, 0, 0, 1, 0, 0, 0, 1],
5 + [1, 2, 0, 0, 0, 1, 0, 4, 0, 1],
6 6 [1, 0, 1, 1, 0, 1, 0, 1, 0, 1],
7 - [1, 0, 1, 0, 0, 0, 0, 1, 0, 1],
7 + [1, 0, 1, 0, 4, 0, 0, 1, 0, 1],
8 8 [1, 0, 1, 0, 1, 1, 0, 1, 0, 1],
9 9 [1, 0, 0, 0, 0, 0, 0, 0, 3, 1],
10 10 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
.....
@@ -14,8 +14,8 @@
14 14 [1, 2, 0, 0, 1, 0, 0, 0, 3, 1],
15 15 [1, 0, 1, 0, 1, 0, 1, 0, 1, 1],
16 16 [1, 0, 1, 0, 0, 0, 1, 0, 0, 1],
17 - [1, 0, 0, 0, 1, 0, 0, 1, 0, 1],
18 - [1, 1, 1, 0, 1, 0, 0, 1, 0, 1],
17 + [1, 0, 4, 0, 1, 0, 0, 1, 0, 1],
18 + [1, 1, 1, 0, 1, 0, 4, 1, 0, 1],
19 19 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
20 20],
21 21 [
.....
+
.....
+



GenAI Prompts

My prompt:

Can you give me the code to generate a new level using arrays or JSON data; using loops to dynamically place obstacles like spikes. Can you also explain your process so I could add a new level if I wanted to: jsconfig.json({ "include": [".js", "libraries/.js", "c:\\Users\\david\\.vscode\\extensions\\wmcicompisci.cs30-p5-1.9.3\\p5types\\global.d.ts"] }) level.js(/* Level.js

A Level represents ONE maze grid loaded from levels.json.

Tile legend (from your original example): 0 = floor 1 = wall 2 = start 3 = goal

Responsibilities:

- Store the grid
- Find the start tile
- Provide collision/meaning queries (isWall, isGoal, inBounds)

- Draw the tiles (including a goal highlight) */

```
class Level { constructor(grid, tileSize) { // Store the tile grid and tile size (pixels per tile).
this.grid = grid; this.ts = tileSize;
```

```
// Start position in grid coordinates (row/col).
// We compute this by scanning for tile value 2.
this.start = this.findStart();
```

```
// Optional: if you don't want the start tile to remain "special"
// after you've used it to spawn the player, you can normalize it
// to floor so it draws like floor and behaves like floor.
if (this.start) {
  this.grid[this.start.r][this.start.c] = 0;
}
```

```
}
```

```
// ----- Size helpers -----
```

```
rows() { return this.grid.length; }
```

```
cols() { return this.grid[0].length; }
```

```
pixelWidth() { return this.cols() * this.ts; }
```

```
pixelHeight() { return this.rows() * this.ts; }
```

```
// ----- Semantic helpers -----
```

```
inBounds(r, c) { return r >= 0 && c >= 0 && r < this.rows() && c < this.cols(); }
```

```
tileAt(r, c) { // Caller should check inBounds first. return this.grid[r][c]; }
```

```
isWall(r, c) { return this.tileAt(r, c) === 1; }
```

```
isGoal(r, c) { return this.tileAt(r, c) === 3; }
```

```
// ----- Start-finding -----
```

```
findStart() { // Scan entire grid to locate the tile value 2 (start). for (let r = 0; r < this.rows();  
r++) { for (let c = 0; c < this.cols(); c++) { if (this.grid[r][c] === 2) { return { r, c }; } } }
```

```
// If a level forgets to include a start tile, return null.
```

```
// (Then the game can choose a default spawn.)
```

```
return null;
```

```
}
```

```
// ----- Drawing -----
```

```
draw() { /* Draw each tile as a rectangle.
```

```
Visual rules (matches your original logic):
```

```
- Walls (1): dark teal
```

```
- Everything else: light floor
```

```
- Goal tile (3): add a highlighted inset rectangle
```

```
*/
```

```
for (let r = 0; r < this.rows(); r++) {  
  for (let c = 0; c < this.cols(); c++) {  
    const v = this.grid[r][c];
```

```
    // Base tile fill
```

```
    if (v === 1) fill(30, 50, 60);
```

```
    else fill(232);
```

```
    rect(c * this.ts, r * this.ts, this.ts, this.ts);
```

```
    // Goal highlight overlay (only on tile 3).
```

```
    if (v === 3) {
```

```
      noStroke();
```

```
      fill(255, 200, 120, 200);
```

```
      rect(
```

```
        c * this.ts + 4,
```

```
        r * this.ts + 4,
```

```
        this.ts - 8,
```

```
        this.ts - 8,
```

```
        6
```

```

    );
  }
}
}

```

```

})) levels.json({ "levels": [ [ [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 2, 0, 0, 0, 1, 0, 0, 0, 1], [1, 0, 1, 1, 0,
1, 0, 1, 0, 1], [1, 0, 1, 0, 0, 0, 0, 1, 0, 1], [1, 0, 1, 0, 1, 1, 0, 1, 0, 1], [1, 0, 0, 0, 0, 0, 0, 0, 3, 1], [1,
1, 1, 1, 1, 1, 1, 1, 1, 1] ], [ [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 2, 0, 0, 1, 0, 0, 0, 3, 1], [1, 0, 1, 0, 1, 0,
1, 0, 1, 1], [1, 0, 1, 0, 0, 0, 1, 0, 0, 1], [1, 0, 0, 0, 1, 0, 0, 1, 0, 1], [1, 1, 1, 0, 1, 0, 0, 1, 0, 1], [1, 1,
1, 1, 1, 1, 1, 1, 1, 1] ] ] }) players.js /* Player.js

```

A Player stores the avatar position in grid coordinates (row/col) and knows how to:

- draw itself
- attempt a move (tile-by-tile) with collision rules

The Player does NOT:

- load JSON
- switch levels Those are "game orchestration" responsibilities that belong in sketch.js. */

```

class Player { constructor(tileSize) { this.ts = tileSize;

```

```

// Current grid position (row/col).

```

```

this.r = 0;

```

```

this.c = 0;

```

```

// Movement throttle (so a key press doesn't move 60 tiles per second).

```

```

this.movedAt = 0;

```

```

this.moveDelay = 90; // ms

```

```

}

```

```

// Place the player at a specific grid location (e.g., the level's start). setCell(r, c) { this.r = r;

```

```

this.c = c; }

```

```

// Convert grid coords to pixel center (for drawing a circle). pixelX() { return this.c * this.ts +
this.ts / 2; }

```

```
pixelY() { return this.r * this.ts + this.ts / 2; }
```

```
draw() { // Same "simple high-contrast avatar" idea as your original. fill(20, 120, 255);  
circle(this.pixelX(), this.pixelY(), this.ts * 0.6); }
```

```
/* Try to move by (dr, dc) tiles.
```

Inputs:

- level: a Level instance, used for bounds + wall collision + goal detection
- dr/dc: desired movement step, typically -1,0,1

Returns:

- true if the move happened
- false if blocked or throttled */ tryMove(level, dr, dc) { // Throttle discrete movement using millis() const now = millis(); if (now - this.movedAt < this.moveDelay) return false;

```
const nr = this.r + dr; const nc = this.c + dc;
```

```
// Prevent walking off the map. if (!level.inBounds(nr, nc)) return false;
```

```
// Prevent walking into walls. if (level.isWall(nr, nc)) return false;
```

```
// Movement is allowed, so commit. this.r = nr; this.c = nc; this.movedAt = now;
```

```
return true; } } ) and sketch.js( const TS = 32;
```

```
// Raw JSON data (from levels.json). let levelsData;
```

```
// Array of Level instances. let levels = [];
```

```
// Current level index. let li = 0;
```

```
// Player instance (tile-based). let player;
```

```
function preload() { // Ensure level data is ready before setup runs. levelsData =  
loadJSON("levels.json"); }
```

```
function setup() { /* Convert raw JSON grids into Level objects. levelsData.levels is an array  
of 2D arrays. */ levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));
```

```

// Create a player. player = new Player(TS);

// Load the first level (sets player start + canvas size). loadLevel(0);

noStroke(); textFont("sans-serif"); textSize(14); }

function draw() { background(240);

// Draw current level then player on top. levels[li].draw(); player.draw();

drawHUD(); }

function drawHUD() { // HUD matches your original idea: show level count and controls.
fill(0); text(Level ${li + 1}/${levels.length} — WASD/Arrows to move, 10, 16); }

function keyPressed() { /* Convert key presses into a movement direction. (WASD + arrows)
*/ let dr = 0; let dc = 0;

if (keyCode === LEFTARROW || key === "a" || key === "A") dc = -1; else if (keyCode ===
RIGHTARROW || key === "d" || key === "D") dc = 1; else if (keyCode === UPARROW || key ===
"w" || key === "W") dr = -1; else if (keyCode === DOWNARROW || key === "s" || key === "S")
dr = 1; else return; // not a movement key

// Try to move. If blocked, nothing happens. const moved = player.tryMove(levels[li], dr, dc);

// If the player moved onto a goal tile, advance levels. if (moved && levels[li].isGoal(player.r,
player.c)) { nextLevel(); } }

// ----- Level switching -----

function loadLevel(idx) { li = idx;

const level = levels[li];

// Place player at the level's start tile (2), if present. if (level.start) {
player.setCell(level.start.r, level.start.c); } else { // Fallback spawn: top-left-ish (but inside
bounds). player.setCell(1, 1); }

// Ensure the canvas matches this level's dimensions. resizeCanvas(level.pixelWidth(),
level.pixelHeight()); }

function nextLevel() { // Wrap around when we reach the last level. const next = (li + 1) %
levels.length; loadLevel(next); }

```

// ----- Utility -----

function copyGrid(grid) { /* Make a deep-ish copy of a 2D array:

- new outer array
- each row becomes a new array

Why copy?

- Because Level constructor may normalize tiles (e.g., replace 2 with 0)
- And we don't want to accidentally mutate the raw JSON data object. */ return grid.map((row) => row.slice()); }

GenAI Prompt

Absolutely—let's add **spikes** as a new tile type and give you two ways to make new levels:

1. **Static** (append arrays to levels.json)
2. **Procedural** (generate a grid in code using loops and simple patterns)

Below I'll show small, drop-in changes to your files and a reusable **level builder** you can call to generate a new level dynamically.

1) Extend your tile legend (add spikes)

We'll use:

- 0 = floor
- 1 = wall
- 2 = start (normalized to 0 after spawn)
- 3 = goal
- 4 = spike (hazard)

Spikes will **reset the player to the start** when stepped on.

2) Minimal changes to your existing files

Level.js — add 4 = spike support

What's new:

- isSpike(r, c) method
- draw a red "spike" overlay on tiles with value 4

Replace your Level.js with this version or merge the "NEW/CHANGED" parts:

JavaScript

/*

Level.js

A Level represents ONE maze grid loaded from levels.json.

Tile legend:

0 = floor

1 = wall

2 = start (normalized to 0 after spawn)

3 = goal

4 = spike (hazard; resets player)

*/

class Level {

```
constructor(grid, tileSize) {

    this.grid = grid;

    this.ts = tileSize;

    this.start = this.findStart();

    // Normalize start tile to floor for drawing/logic once we captured it.

    if (this.start) {

        this.grid[this.start.r][this.start.c] = 0;

    }

}

// ----- Size helpers -----

rows() { return this.grid.length; }

cols() { return this.grid[0].length; }

pixelWidth() { return this.cols() * this.ts; }

pixelHeight() { return this.rows() * this.ts; }

// ----- Semantic helpers -----

inBounds(r, c) {

    return r >= 0 && c >= 0 && r < this.rows() && c < this.cols();
```

```
}
```

```
tileAt(r, c) {
```

```
  return this.grid[r][c];
```

```
}
```

```
isWall(r, c) { return this.tileAt(r, c) === 1; }
```

```
isGoal(r, c) { return this.tileAt(r, c) === 3; }
```

```
// NEW: hazards
```

```
isSpike(r, c) { return this.tileAt(r, c) === 4; }
```

```
// ----- Start-finding -----
```

```
findStart() {
```

```
  for (let r = 0; r < this.rows(); r++) {
```

```
    for (let c = 0; c < this.cols(); c++) {
```

```
      if (this.grid[r][c] === 2) {
```

```
        return { r, c };
```

```
      }
```

```
    }
```

```
  }
```

```
return null;
```

```
}
```

```
// ----- Drawing -----
```

```
draw() {
```

```
  for (let r = 0; r < this.rows(); r++) {
```

```
    for (let c = 0; c < this.cols(); c++) {
```

```
      const v = this.grid[r][c];
```

```
      // Base tile
```

```
      if (v === 1) fill(30, 50, 60); // wall
```

```
      else fill(232); // floor & others
```

```
      rect(c * this.ts, r * this.ts, this.ts, this.ts);
```

```
      // Goal highlight overlay
```

```
      if (v === 3) {
```

```
        noStroke();
```

```
        fill(255, 200, 120, 200);
```

```
        rect(
```

```
          c * this.ts + 4,
```

```
r * this.ts + 4,
```

```
this.ts - 8,
```

```
this.ts - 8,
```

```
6
```

```
);
```

```
}
```

```
// NEW: Spike overlay (simple red "teeth")
```

```
if (v === 4) {
```

```
  const x = c * this.ts;
```

```
  const y = r * this.ts;
```

```
  noStroke();
```

```
  fill(220, 50, 50);
```

```
// draw 3 small triangles along the bottom edge
```

```
const teeth = 3;
```

```
const w = this.ts / teeth;
```

```
for (let i = 0; i < teeth; i++) {
```

```
  const x0 = x + i * w;
```

```

triangle(

x0, y + this.ts,

x0 + w, y + this.ts,

x0 + w / 2, y + this.ts - this.ts * 0.65

);

}

}

}

}

}

}

}

```

Show more lines

sketch.js — reset on spikes + add a procedurally generated level

What's new:

- After a move, if player stands on a spike → `loadLevel(li)` to reset the level.
- A **procedural level builder** (`buildProceduralLevel(...)`) that uses loops to place spikes in patterns.
- We append the generated level to your levels array after loading JSON levels.

Replace your sketch.js with this version or merge the "NEW/CHANGED" parts:

JavaScript

```
const TS = 32;
```

```
let levelsData; // from levels.json
```

```
let levels = []; // array of Level instances
```

```
let li = 0; // current level index
```

```
let player;
```

```
function preload() {
```

```
  levelsData = loadJSON("levels.json");
```

```
}
```

```
function setup() {
```

```
  // Convert raw JSON to Level objects
```

```
  levels = levelsData.levels.map((grid) => new Level(copyGrid(grid), TS));
```

```
  // NEW: Append a procedurally generated level
```

```
  const procGrid = buildProceduralLevel(12, 18, 424242);
```

```
  levels.push(new Level(copyGrid(procGrid), TS));
```

```
  player = new Player(TS);
```

```
  loadLevel(0);
```

```
noStroke();
```

```
textFont("sans-serif");
```

```
textSize(14);
```

```
}
```

```
function draw() {
```

```
  background(240);
```

```
  levels[li].draw();
```

```
  player.draw();
```

```
  drawHUD();
```

```
}
```

```
function drawHUD() {
```

```
  fill(0);
```

```
  text(`Level ${li + 1}/${levels.length} — WASD/Arrows to move`, 10, 16);
```

```
  text(`Spikes reset you`, 10, 34); // NEW: small hint
```

```
}
```

```
function keyPressed() {
```

```
let dr = 0, dc = 0;

if (keyCode === LEFT_ARROW || key === "a" || key === "A") dc = -1;

else if (keyCode === RIGHT_ARROW || key === "d" || key === "D") dc = 1;

else if (keyCode === UP_ARROW || key === "w" || key === "W") dr = -1;

else if (keyCode === DOWN_ARROW || key === "s" || key === "S") dr = 1;

else return;

const moved = player.tryMove(levels[li], dr, dc);

// NEW: check hazard first

if (moved && levels[li].isSpike(player.r, player.c)) {

loadLevel(li); // reset to the level start

return;

}

// Goal check

if (moved && levels[li].isGoal(player.r, player.c)) {

nextLevel();

}

}
```

```
// ----- Level switching -----
```

```
function loadLevel(idx) {
```

```
  li = idx;
```

```
  const level = levels[li];
```

```
  if (level.start) player.setCell(level.start.r, level.start.c);
```

```
  else player.setCell(1, 1);
```

```
  resizeCanvas(level.pixelWidth(), level.pixelHeight());
```

```
}
```

```
function nextLevel() {
```

```
  const next = (li + 1) % levels.length;
```

```
  loadLevel(next);
```

```
}
```

```
// ----- Utility -----
```

```
function copyGrid(grid) {
```

```
  return grid.map((row) => row.slice());
```

```
}
```

```
/* -----
```

NEW: A small procedural level builder using loops and patterns.

Returns a 2D array of numbers using the same tile legend:

0=floor, 1=wall, 2=start, 3=goal, 4=spike

```
-----*/
```

```
function buildProceduralLevel(rows, cols, seed = 12345) {
```

```
// Safety: minimum viable size with outer walls
```

```
rows = Math.max(rows, 7);
```

```
cols = Math.max(cols, 10);
```

```
// 1) Start with all walls
```

```
const G = Array.from({ length: rows }, () => Array(cols).fill(1));
```

```
// 2) Carve interior floors with a double loop
```

```
for (let r = 1; r < rows - 1; r++) {
```

```
  for (let c = 1; c < cols - 1; c++) {
```

```
    G[r][c] = 0; // floor
```

```
  }
```

```
}
```

```
// 3) Place start & goal
```

```
const start = { r: 1, c: 1 };
```

```
const goal = { r: rows - 2, c: cols - 2 };
```

```
G[start.r][start.c] = 2;
```

```
G[goal.r][goal.c] = 3;
```

```
// 4) Add some internal wall "pillars" using loops
```

```
// (simple columns to create navigation choices)
```

```
for (let r = 2; r < rows - 2; r++) {
```

```
  if (r % 2 === 0) {
```

```
    const c1 = Math.floor(cols * 0.33);
```

```
    const c2 = Math.floor(cols * 0.66);
```

```
    G[r][c1] = 1;
```

```
    G[r][c2] = 1;
```

```
  }
```

```
}
```

```
// 5) Spike patterns with loops
```

```
// 5a) Checkerboard spikes in a band (every 2 tiles)
```

```
for (let r = 2; r < rows - 2; r++) {
```

```
  for (let c = 2; c < cols - 2; c++) {
```

```
    if ((r + c) % 2 === 0 && r >= Math.floor(rows * 0.35) && r <= Math.floor(rows * 0.65)) {
```

```
      if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;
```

```
    }
```

```
  }
```

```
}
```

```
// 5b) A vertical "spike line" on every 3rd column
```

```
for (let c = 3; c < cols - 3; c += 3) {
```

```
  for (let r = 2; r < rows - 2; r++) {
```

```
    if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) G[r][c] = 4;
```

```
  }
```

```
}
```

```
// 5c) Sprinkle some random spikes (deterministic with a seed)
```

```
const rnd = mulberry32(seed);
```

```
let sprinkled = Math.floor((rows * cols) * 0.08);
```

```
while (sprinkled > 0) {

  const r = 1 + Math.floor(rnd() * (rows - 2));

  const c = 1 + Math.floor(rnd() * (cols - 2));

  if (!isStartOrGoal(start, goal, r, c) && G[r][c] === 0) {

    G[r][c] = 4;

    sprinkled--;

  }

}

// Optional: ensure at least a horizontal corridor from start side

// (reduce frustration; keeps a guaranteed safe band)

const safeRow = start.r + 1;

for (let c = start.c; c < cols - 1; c++) {

  if (!isStartOrGoal(start, goal, safeRow, c) && G[safeRow][c] !== 1) {

    G[safeRow][c] = 0; // clear any spikes from this band

  }

}

return G;
```

```
}
```

```
function isStartOrGoal(start, goal, r, c) {
```

```
  return (r === start.r && c === start.c) || (r === goal.r && c === goal.c);
```

```
}
```

```
// Deterministic RNG for reproducible procedural levels
```

```
function mulberry32(a) {
```

```
  return function () {
```

```
    let t = a += 0x6D2B79F5;
```

```
    t = Math.imul(t ^ (t >>> 15), t | 1);
```

```
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
```

```
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
```

```
  };
```

```
}
```

Show more lines

✅ With these changes, you now have **all your JSON levels** plus one **generated** level at the end.
Use seed to get repeatable spike layouts.

players.js — no changes required

Your move/try logic is good. We do hazard resolution in sketch.js after a move completes.

3) Option A: Add a new level in levels.json (static array)

You can manually add levels that include spikes by appending another 2D array to levels.levels.

Example:

JSON

{

"levels": [

/* (existing levels here) */

[

[1,1,1,1,1,1,1,1,1,1],

[1,2,0,0,0,0,4,0,3,1],

[1,0,1,4,1,0,1,0,1,1],

[1,0,1,0,0,0,1,4,0,1],

[1,0,0,0,1,0,0,1,0,1],

[1,1,1,0,1,4,0,1,0,1],

[1,1,1,1,1,1,1,1,1,1]

]

]

```
}
```

Show more lines

Note: 4s are spikes. Make sure there's at least one path to the 3 (goal).

4) Option B: Add levels procedurally (dynamic, with loops)

You already saw `buildProceduralLevel()`. If you want **fine-grained control** over placement patterns, here are small helper functions you can drop into `sketch.js` and call from your builder:

JavaScript

// Places spikes every N tiles in a rectangular region.

```
function placeSpikesEveryN(G, r0, c0, r1, c1, step = 2) {
```

```
  for (let r = r0; r <= r1; r += step) {
```

```
    for (let c = c0; c <= c1; c += step) {
```

```
      if (G[r][c] === 0) G[r][c] = 4;
```

```
    }
```

```
  }
```

```
}
```

// Places a horizontal OR vertical spike line (dir = 'h' | 'v')

```
function placeSpikeLine(G, r, c, length, dir = 'h') {
```

```
  for (let i = 0; i < length; i++) {
```

```

const rr = dir === 'h' ? r : r + i;

const cc = dir === 'h' ? c + i : c;

if (rr >= 0 && cc >= 0 && rr < G.length && cc < G[0].length) {

  if (G[rr][cc] === 0) G[rr][cc] = 4;

}

}

}

```

// Places a rectangular border of walls or spikes

```

function placeRectBorder(G, r0, c0, r1, c1, tileValue = 1) {

  for (let c = c0; c <= c1; c++) {

    G[r0][c] = tileValue;

    G[r1][c] = tileValue;

  }

  for (let r = r0; r <= r1; r++) {

    G[r][c0] = tileValue;

    G[r][c1] = tileValue;

  }

}

```

..

Show more lines

You can call these inside your builder with loops to craft unique patterns quickly.

5) How to add a new level—step by step

Approach A: JSON file

1. Open levels.json.
2. Append a new 2D array to the "levels" array.
3. Use tile codes (0,1,2,3,4). Ensure one 2 (start) and one 3 (goal).
4. Save the file. Reload the sketch—your new level will appear after the others.

Approach B: In code (procedural)

1. In setup(), after loading JSON levels, build your grid:

JavaScript

```
const myGrid = buildProceduralLevel(14, 22, 9001);
```

```
levels.push(new Level(copyGrid(myGrid), TS));
```

Show more lines

2. Tweak buildProceduralLevel() (loops/patterns) to change shape and difficulty:
 - a. Use **every-N** placement
 - b. Lines, rings, checkerboards
 - c. Random sprinkle with a seed for reproducibility

6) Tips to avoid unwinnable levels

- Keep at least one **safe corridor** (as shown with safeRow).

- Or, after generating spikes, run a simple **BFS path check** from start to goal treating spikes as blocked tiles. If no path, remove some spikes and try again. (I can add that for you if you want.)

7) What would you like next?

- Do you want spikes to **toggle on/off** on a timer?
- Should stepping on spikes **remove a life** instead of a full reset?
- Want me to add a **path validator** so generated levels always remain solvable?

If you tell me your preferred spike behavior and difficulty curve, I can tailor the generator and add safeguards.